You said:

codigo(1).c
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 45s

ChatGPT said:

Análise de segurança do arquivo `codigo(1).c`

Veredito: MALWARE

Tipo de arquivo: código-fonte C · Método: análise estática, sem execução

Categoria principal: bot de DDoS controlado remotamente, com funcionalidades de backdoor e execução de comandos via IRC.

O código apresenta características fortemente associadas à família **Kaiten/Tsunami**: conecta-se a servidores IRC, aguarda comandos remotos e implementa diferentes métodos de inundação de tráfego para ataques de negação de serviço. A identificação é sustentada tanto pelas funções encontradas no arquivo quanto por análises de segurança dessa família de malware. Trend Micro ... +2

1. Identificação inicial e configuração

Nas primeiras linhas, o programa define o nome que pretende aparentar ter no sistema, o canal IRC e uma chave:

```
#define FAKENAME "-bash"
#define CHAN "#whatever"
#define KEY "bleh"
```

Também contém uma lista de quatro servidores IRC:

```
char *servers[] = {
    "irc.qeast.net",
    "irc2.magic.ca",
    "irc.east.gblx.net",
    "irc.west.gblx.net",
```

```
    (void*)0
};
```

Por que isso é suspeito?

O uso de um nome que imita um processo legítimo, combinado com servidores de controle remotos definidos no código, é compatível com um agente que procura ocultar sua presença e receber instruções de um operador.

É importante observar que `#whatever`, `bleh` e os endereços dos servidores estão presentes no código analisado, mas não é possível concluir apenas por isso que esses servidores ainda existam ou estejam operacionais.

2. Inicialização e tentativa de ocultação

Na função `main()`, o programa executa:

```
if (fork()) exit(0);
```

Esse padrão cria um processo filho e faz o processo original terminar, permitindo que o filho continue em segundo plano.

Depois, o programa modifica o nome do processo:

```
strncpy(argv[0], FAKENAME, strlen(argv[0]));
```

E limpa os argumentos adicionais da linha de comando:

```
for (on=1;on<argc;on++)
    memset(argv[on],0,strlen(argv[on]));
```

Essas operações são compatíveis com uma tentativa de dificultar a identificação do processo em ferramentas administrativas.

Ressalva: a alteração de `argv[0]` não garante que o programa fique invisível. Além disso, o código contém possíveis problemas de segurança de memória e a eficácia da técnica depende do ambiente.

3. Comunicação com o controlador remoto (IRC)

A função `con()` estabelece conexão TCP com um servidor selecionado da lista de configuração:

```
if (changeservers == 0)
    server = servers[rand()%numservers];
```

Em seguida, conecta-se à porta 6667:

```
srv.sin_port = htons(6667);
```

Após conectar, o processo envia os comandos IRC de identificação:

```
Send(sock,"NICK %s\nUSER %s localhost localhost :%s\n",
     nick,ident,user);
```

A função `_376()` entra no canal configurado e consulta informações sobre o próprio cliente. A função `_PING()` responde aos pedidos de ping do servidor.

O tratamento das mensagens recebidas ocorre principalmente em `_PRIVMSG()`. O programa identifica mensagens dirigidas ao canal configurado e interpreta comandos prefixados por `!`.

Esse mecanismo permite que um operador remoto controle o comportamento do processo por meio do IRC, uma característica típica de um **bot controlado por comando e controle (C2)**. Trend Micro ... +1

4. Funcionalidades de ataque DDoS

Esta é a evidência mais forte de intenção maliciosa. O arquivo implementa diferentes funções para gerar tráfego contra um alvo escolhido remotamente.

Função	Comportamento observado
`udp()`	Envia pacotes UDP usando um socket de baixo nível para gerar tráfego em grande volume.
`pan()`	Constrói pacotes TCP com a flag SYN ativada, compatíveis com um ataque de inundação SYN.

`tsunami()`	Constrói e envia pacotes TCP repetidamente para um destino especificado.
`unknown()`	Envia datagramas UDP de grande tamanho a portas aleatórias do alvo.

As funções de ataque criam loops de envio contínuo e, em alguns casos, recebem uma duração definida pelo operador.

Por exemplo, em `udp()`:

```
sendto(get,buf,1500,0,
       (struct sockaddr *)&s_in,sizeof(s_in));
```

A chamada envia o pacote construído ao endereço de destino. O processo repete essa operação para gerar tráfego, usando também endereços de origem falsificados em determinadas configurações.

A função `pan()` constrói pacotes TCP com SYN ativado, enquanto `tsunami()` monta outro tipo de tráfego TCP para inundação. A finalidade dessas rotinas não é apenas testar conectividade: o código também descreve explicitamente as funções como ferramentas de flooding na rotina `help()`.

Essas características coincidem com as funcionalidades de DDoS documentadas para Kaiten/Tsunami. Trend Micro ... +1

5. Falsificação de endereços IP

As funções `spoof()`, `getspoof()` e `getspoofs()` permitem configurar e consultar endereços IP de origem usados na geração de pacotes.

O código de `spoof()` aceita uma faixa de endereços e calcula quantos endereços deverão ser utilizados. Já `getspoof()` escolhe um endereço dessa configuração.

Nas funções de ataque, esse valor é aplicado ao campo de endereço IP de origem, por exemplo:

```
ip->saddr = getspoof();
```

A falsificação de endereços de origem pode dificultar a identificação da origem real dos pacotes e é uma funcionalidade compatível com ataques de negação de serviço.

6. Execução remota de comandos

Além dos ataques DDoS, o código oferece recursos que tornam o programa uma backdoor.

Comando `SH`

Em `_PRIVMSG()`, o programa trata a instrução `SH` utilizando `popen()`:

```
command = popen(buf,"r");
```

O conteúdo recebido do controlador é incorporado ao comando executado pelo shell. A saída da execução é então encaminhada ao operador por mensagens IRC.

Isso permite executar comandos arbitrários no contexto de permissões do processo, sujeito às limitações do sistema.

Comando `GET`

A função `get()` abre uma conexão HTTP, solicita um recurso remoto e grava o conteúdo em um arquivo local:

```
file = fopen(argv[2],"wb");
```

O nome e o endereço do arquivo são fornecidos pelo comando remoto. Assim, o operador pode baixar arquivos adicionais para o sistema comprometido.

O programa também implementa os comandos `IRC`, para encaminhar comandos ao servidor, e `SERVER`, para alterar o servidor IRC utilizado.

A combinação de execução de shell e download remoto representa uma capacidade de controle bem mais ampla que a de um simples gerador de tráfego.

7. Ocultação, controle do processo e persistência

O código possui mecanismos adicionais:

- `FAKENAME`: tenta apresentar o processo com o nome `-bash`.
- `fork()`: faz o processo continuar em segundo plano.
- `KILLALL`: tenta encerrar os processos filhos registrados.

- `DISABLE` e `ENABLE`: implementam controles para suspender e reativar as operações de ataque.
- `NICK`: permite mudar o apelido usado na conexão IRC.

Há, porém, duas ressalvas importantes. O código contém blocos condicionais para persistência e um serviço de identificação (`identd`), mas as macros `STARTUP` e `IDENT` são explicitamente desativadas no início do arquivo por meio de `#undef` . Portanto, essas funcionalidades não estão habilitadas nessa configuração do código-fonte.

Também há erros de implementação, como possíveis estouros de memória e problemas em alguns cálculos dos pacotes. Isso pode impedir certas operações de funcionarem corretamente, mas não altera a finalidade maliciosa evidenciada pelas demais funções.

8. Avaliação das evidências

Critério	Resultado
Comunicação com controlador remoto	Presente
Recebimento de comandos remotos	Presente
Execução de comandos do sistema	Presente
Download de arquivos	Presente
Geração de tráfego para DDoS	Presente
Falsificação de IP	Presente
Tentativa de ocultação do processo	Presente
Persistência automática habilitada nesta configuração	Não demonstrada

O programa ainda se identifica com a mensagem `Kaiten wa goraku` , na função `version()` . Essa identificação textual, somada aos nomes e comportamentos das funções, reforça a correspondência com o código associado à família Kaiten/Tsunami.

A análise é estática: não foi feita execução do programa nem verificação de conectividade dos servidores configurados. Portanto, a conclusão se refere ao comportamento implementado no código, não à confirmação de que todas as rotinas funcionariam corretamente em um ambiente específico.

Conclusão final

Arquivo classificado como malware

Classificação principal: bot de DDoS controlado remotamente via IRC, com funcionalidades de backdoor.

Família compatível: Kaiten/Tsunami.

Confiança da classificação: alta.

As evidências decisivas são a comunicação com um controlador IRC, os comandos remotos de execução de shell e download de arquivos e as rotinas específicas de inundação UDP e TCP. Não se trata de um programa benigno com comportamento incidentalmente suspeito.

Recomendação: não compile nem execute esse código em um sistema de uso pessoal ou de produção. Se ele já tiver sido executado em uma máquina, investigue as conexões de saída, os processos e os comandos que possam ter sido executados remotamente.

Também posso criar um ↪ diagrama visual do malware.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal deste malware é **Bot de DDoS (Distributed Denial of Service)**, controlado remotamente via IRC.

Família provável: Kaiten/Tsunami.

Ele também possui funcionalidades de backdoor, incluindo execução remota de comandos e download de arquivos.